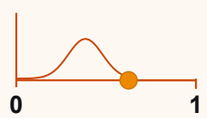
1) ENCODING: build an ordinal key and its nested tail word

Scalar input x



one channel-pixel,
$x \in [0, 1]$

Ordered thresholds



$K - 1$ strict cuts
define ordinal class

Ordinal key k



$k \in \{0, \ldots, K-1\}$
available at encoding

Cumulative tail word P_k



one of K nested
$(K-1)$-step words

Example OQE codebook (K=7)

	t_1	t_2	t_3	t_4	t_5	t_6
k=0	0	0	0	0	0	0
k=1	1	0	0	0	0	0
k=2	1	1	0	0	0	0
k=3	1	1	1	0	0	0
k=4	1	1	1	1	0	0
k=5	1	1	1	1	1	0
k=6	1	1	1	1	1	1

$k = \sum_t P_k(t)$ one-bit nested chain

Two structural properties of OQE

- **Ordered event count**
  $||P_k||_0 = k$
  count recovery / deletion law
- **Nested support**
  grouped currents become suffix sums

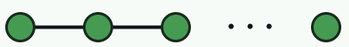
2) ORDINARY INFERENCE PATH

Hybrid input $[x, x, P_k]$



two analog anchors + binary tail

First linear layer



Downstream SNN



conventional spiking backbone
and training

3) GROUPED EXECUTION PATH

Routes grouped by k



same complete tail word;
no sequence matching

Class aggregates

$$A_k = \sum_{r:\, k_r = k} W_r$$

sum of weights over
all routes with key k

Suffix-structured execution (canonical OQE)

$$I_t = \sum_{k>t} A_k$$
$$I_t = I_{t+1} + A_{t+1}$$

compute suffix sums via
reverse scan over k

Downstream SNN



conventional spiking backbone
and training

Post-grouping reconstruction $O(K^2)$ generic class–time relation $\longrightarrow$ $O(K)$ reverse scan